

CAPÍTULO 9

ESTRUCTURAS Y UNIONES

CONTENIDO

- | | |
|--|---------------------------------|
| 9.1. Estructuras. | 9.6. Uniones. |
| 9.2. Acceso a estructuras. | 9.7. Enumeraciones. |
| 9.3. Estructuras anidadas. | 9.8. Campos de bit. |
| 9.4. Arrays de estructuras. | 9.9. <i>Resumen.</i> |
| 9.5. Utilización de estructuras
como parámetros. | 9.10. <i>Ejercicios.</i> |
| | 9.11. <i>Problemas.</i> |

INTRODUCCIÓN

Este capítulo examina estructuras, uniones, enumeraciones y tipos definidos por el usuario que permite a un programador crear nuevos tipos de datos. La capacidad para **crear** nuevos tipos es una característica importante y potente de C y libera a un programador de restringirse al uso de los tipos ofrecidos por el lenguaje. Una **estructura** contiene múltiples variables, que pueden ser de tipos diferentes. La estructura es importante para la creación de programas potentes, tales como bases de datos u otras aplicaciones que requieran grandes cantidades de datos. Por otra parte, se analizará el concepto de **unión**, otro tipo de dato no tan importante como las estructuras array y estructura, pero si necesarias en algunos casos.

Un tipo de dato enumerado es una colección de miembros con nombre que tienen valores enteros equivalentes. Un **typedef** es de hecho no un nuevo tipo de dato sino simplemente un sinónimo de un tipo existente.

CONCEPTOS CLAVE

- Estructura.
- Estructuras anidadas.
- Selector de campos.
- **struct**.
- **sizeof**.
- **union**.
- **typedef**.
- Operadores de bits,

9.1. ESTRUCTURAS

Los arrays son estructuras de datos que contienen un número determinado de elementos (**su tamaño**) y todos los elementos han de ser del mismo tipo de datos; es una estructura de datos homogénea. Esta característica supone una gran limitación cuando se requieren grupos de elementos con tipos diferentes de datos cada uno. Por ejemplo, si se dispone de una lista de temperaturas, es muy útil un array; sin embargo, si se necesita una lista de información de clientes que contengan elementos tales como el nombre, la edad, la dirección, el número de la cuenta, etc., los arrays no son adecuados. La solución a este problema es utilizar un tipo de dato registro, en C llamado **estructura**.

Los componentes individuales de una estructura se llaman **miembros**. Cada miembro (elemento) de una estructura puede contener datos de un tipo diferente de otros miembros. Por ejemplo, se puede utilizar una estructura para almacenar diferentes tipos de información sobre una persona, tal como nombre, estado civil, edad y fecha de nacimiento. Cada uno de estos elementos se denominan **nombre del miembro**.

Una estructura es una colección de uno o más tipos de elementos denominados miembros, cada uno de los cuales puede ser un tipo de dato diferente.

Una estructura puede contener cualquier número de miembros, cada uno de los cuales tiene un nombre único, denominado **nombre** del miembro. Supongamos que se desea almacenar los datos de una colección de discos compactos (**CD**) de música. Estos datos pueden ser:

- Título.
- Artista.
- Número de canciones.
- Precio.
- Fecha de compra.

La estructura CD contiene cinco miembros. Tras decidir los miembros, se debe decidir cuáles son los tipos de datos para utilizar por los miembros. Esta información se representa en la tabla siguiente:

Nombre miembro	Tipo de dato
Título	Array de caracteres de tamaño 30 .
Artista	Array de caracteres de tamaño 25.
Número de canciones	Entero.
Precio	Coma flotante.
Fecha de compra	Array de caracteres de tamaño 8.

La Figura 9.1 contiene la estructura CD, mostrando gráficamente los tipos de datos dentro de la estructura. Obsérvese que cada miembro es un tipo de dato diferente.

Título	Ay, ay, ay, cómo se aleja el sol.
Artista	No me pises la sandalias.
Número de canciones	10
Precio	2222.25
Fecha de compra	8-10-1999

Figura 9.1. Representación gráfica de una estructura CD.

9.1.1. Declaración de una estructura

Una estructura es un tipo de dato definido por el usuario, que se debe declarar antes de que se pueda utilizar. El formato de la declaración es:

```
struct <nombre de la estructura>
{
    <tipo de dato miembro> <nombre miembro>
    <tipo de dato miembro> <nombre miembro>
    ...
    <tipo de dato miembro> <nombre miembro>
};
```

La declaración de la estructura CD es

```
struct coleccion_CD
{
    char titulo[30];
    char artista[25];
    int num-canciones;
    float precio;
    char fecha-compra[8];
};
```

Ejemplo

```
struct complejo
{
    float parte-real, parte-imaginaria;
};
```

En este otro ejemplo se declara el tipo estructura venta ;

```
struct venta
{
    char vededor[30];
    unsigned int codigo;
    int inids_articulos;
    float precio-unit;
};
```

9.1.2. Definición de variables de estructuras

Al igual que a los tipos de datos enumerados, a una estructura se accede utilizando una variable o variables que se deben definir después de la declaración de la estructura. Del mismo modo que sucede en otras situaciones, en C existen dos conceptos similares a considerar, *declaración* y *definición*. La diferencia técnica es la siguiente, una declaración especifica simplemente el nombre y el formato de la estructura de datos, pero no reserva almacenamiento en memoria; la declaración especifica un nuevo tipo de dato: `struct <nombre-estructura>`. Por consiguiente, cada definición de variable para una estructura dada crea un área en memoria en donde los datos se almacenan de acuerdo al formato estructurado declarado.

Las variables de estructuras se pueden definir de dos formas: 1) listándolas inmediatamente después de la llave de cierre de la declaración de la estructura, o 2) listando el tipo de la estructura creado seguida por las variables correspondientes en cualquier lugar del programa antes de utilizarlas. La definición y declaración de la estructura `colecciones-CD` se puede hacer por cualquiera de los dos métodos:

```

1. struct colecciones-CD
{
    char titulo[30];
    char artista[25];
    int num-canciones;
    float precio;
    char fecha_compra[8];
} cd1, cd2, cd3;

2. struct colecciones_CD cd1, cd2, cd3;

```

Otros ejemplos de definición/declaración

Considérese un programa que gestione libros y procese los siguientes datos: título del libro, nombre del autor, editorial y año de publicación. Una estructura `info-libro` podría ser:

```

struct info_libro
{
    char titulo[60];
    char autor[30];
    char editorial[30];
    int anyo;
};

```

La definición de la estructura se puede hacer así:

```

1. struct info_libro
{
    char titulo[60];
    char autor[30];
    char editorial[30];
    int anyo;
} librol, libro2, libro3;

2. struct info_libro librol, libro2, libro3;

```

Ahora se nos plantea una aplicación de control de los participantes en una carrera popular, cada participante se representa por los datos: nombre, edad, sexo, categoría, club y tiempo. El registro se representa con la estructura `corredor`:

```

struct corredor
{
    char nombre[40];
    int edad;
    char sexo;
    char categoria[20];
    char club[26];
    float tiempo;
};

```

La definición de variables estructura se puede hacer así:

```

struct corredor v1, s1, c1;

```

9.1.3. Uso de estructuras en asignaciones

Como una estructura es un tipo de dato similar a un `int` o un `char`, se puede asignar una estructura a otra. Por ejemplo, se puede hacer que `libro3`, `libro4` y `libro5` tengan los mismos valores en sus miembros que `libro1`. Por consiguiente, sería necesario realizar las siguientes sentencias:

```

libro3 = librol;
libro4 = librol;
libro5 = librol;

```

De modo alternativo se puede escribir

```

libro4 = libro5 = libro6 = librol;

```

9.1.4. Inicialización de una declaración de estructuras

Se puede inicializar una estructura de dos formas. Se puede inicializar una estructura dentro de la sección de código de su programa, o bien se puede inicializar la estructura como parte de la definición. Cuando se inicializa una estructura como parte de la definición, se especifican los valores iniciales, entre llaves, después de la definición de variables estructura. El formato general en este caso:

```

struct <tipo> <nombre variable estructura> =
{ valor miembro,
  valor miembro ,
  ...
  valor miembro
};

struct info-libro
{
  char titulo[60];
  char auto[30];
  char editorial[30];
  int anyo;
} librol = {"Maravilla del saber ", "Lucas Garcia", "McGraw-Hill", 1999};

```

Otro ejemplo podría ser:

```

struct coleccion_CD
{
  char titulo[30];
  char artista[25];
  int num-canciones;
  float precio;
  char fecha_compra[8];
} cd1 = {
    "El humo nubla tus ojos",
    "Col Porter",
    15,
    2545,
    "02/ 6/ 99"
};

```

Otro ejemplo con la estructura corredor:

```

struct corredor vl = {
    "Salvador Rapido",
    29,
    'V',
    "Senior",
    'Independiente',
    0.0
};

```

9.1.5. El tamaño de una estructura

El operador `sizeof` se aplica sobre un tipo de datos, o bien sobre una variable. Se puede aplicar para determinar el tamaño que ocupa en memoria una estructura. El siguiente programa ilustra el uso del operador `sizeof` para determinar el tamaño de una estructura:

```
#include <stdio.h>

/* declarar una estructura Persona */
struct persona
{
    char nombre[30];
    int edad;
    float altura;
    float peso;
};

void main()
{
    struct persona mar;
    printf("Sizeof (persona): %d \n", sizeof (mar)) ;
}
```

Al ejecutar el programa se produce la salida:

Sizeof (persona) : 40

El resultado se obtiene determinando el número de bytes que ocupa la estructura

Persona	Miembros dato	Tamaño (bytes)
nombre[30]	char(1)	30
edad	int(2)	2
altura	float(4)	4
peso	float(4)	4
<i>Total</i>		40

9.2. ACCESO A ESTRUCTURAS

Cuando se accede a una estructura, o bien se almacena información en la estructura o se recupera la información de la estructura. Se puede acceder a los miembros de una estructura de una de estas dos formas: 1) utilizando el operador punto (`.`), o bien 2) utilizando el operador puntero `->`.

9.2.1. Almacenamiento de información en estructuras

Se puede almacenar información en una estructura mediante inicialización, asignación directa o lectura del teclado. El proceso de inicialización ya se ha examinado, veamos ahora la asignación directa y la lectura del teclado.

Acceso a una estructura de datos mediante el operador punto

La asignación de datos a los miembros de una variable estructura se hace mediante el operador punto. La sintaxis en C es:

```
<nombre variable estructura> . <nombre miembro> = datos;
```

Algunos ejemplos:

```
strcpy(cd1.titulo, "Granada");
cd1.precio = 3450.75;
cd1.num_canciones = 7;
```

El operador punto proporciona el camino directo al miembro correspondiente. Los datos que se almacenan en un miembro dado deben ser del mismo tipo que el tipo declarado para ese miembro. En el siguiente ejemplo se lee del teclado los datos de una variable estructura corredor:

```
struct corredor cr;

printf("Nombre: ");
gets(cr.nombre);
printf("edad: ");
scanf("%d",&cr.edad);
printf("Sexo: ");
scanf("%c",&cr.sexo);
printf("Club: ");
gets(cr.club);
if (cr.edad <= 18)
    cr.categoria = "Juvenil";
elseif (cr.edad <= 40)
    cr.categoria = "Senior";
else
    cr.categoria = "Veterano";
```



Acceso a una estructura de datos mediante el operador puntero

El operador puntero, `->`, sirve para acceder a los datos de la estructura a partir de un puntero. Para utilizar este operador se debe definir primero una variable puntero para apuntar a la estructura. A continuación, utilice simplemente el operador puntero para apuntar a un miembro dado.

La asignación de datos a estructuras utilizando el operador puntero tiene el formato:

```
<puntero estructura> -> <nombre miembro> = datos;
```

Así, por ejemplo, una estructura estudiante

```
struct estudiante
{
    char Nombre[41];
    int Num_Estudiente;
    int Anyo-de-matricula;
    float Nota;
};
```

Se puede definir `ptr_est` como un puntero a la estructura

```
struct estudiante *ptr_est;
struct estudiante mejor;
```

A los miembros de la estructura estudiante se pueden asignar datos como sigue (siempre y cuando la estructura ya tenga creado su espacio de almacenamiento, por ejemplo, con `malloc()`; o bien, tenga la dirección de una variable estructura).

```
ptr_est = &mejor; /* ptr_est tiene la dirección(apunta a) mejor */
strcpy(ptr_est->Nombre, "Pepe alomdra");
ptr_est -> Num_Estudiente = 3425;
ptr_est -> Nota = 8.5;
```


Nota

Previamente habría que crear espacio de almacenamiento en memoria; por ejemplo, con la función `malloc()`.

9.2.2. Lectura de información de una estructura

Si ahora se desea introducir la información en la estructura basta con acceder a los miembros de la estructura con el operador punto o flecha (puntero). Se puede introducir la información desde el teclado o desde un archivo, o asignar valores calculados.

Así, si `z` es una variable de tipo estructura complejo, se lee parte real, parte imaginaria y se calcula el módulo:

```
struct complejo
{
    float pr;
    float pi;
    float modulo;
};
struct complejo z;

printf("\nParte real: ");
scanf("%f",&z.pr);
printf("\nParte imaginaria: ");
scanf("%f",&z.pi);
/* calculo del módulo */
z.modulo = sqrt(z.pr*z.pr + z.pi*z.pi);
```

9.2.3. Recuperación de información de una estructura

Se recupera información de una estructura utilizando el operador de asignación o una sentencia de salida (`printf()`, `puts()`, ...). Igual que antes, se puede emplear el operador punto o el operador flecha (puntero). El formato general toma uno de estos formatos:

1. `<nombre variable> =`
 `<nombre variable estructura>.<nombre miembro>;`

o bien

`<nombre variable> =`
 `<puntero de estructura> -> <nombre miembro>;`

2 .para salida:

`printf(" ",<nombre variable estructura>.<nombre miembro>;`

o bien

`printf(" ",<puntero de estructura>-> <nombre miembro>;`

Algunos ejemplos del uso de la estructura complejo:

```
float x,y;
struct complejo z;
struct complejo *pz;
```

```

pz = &z;

x = z.pr;
y = z.pi;
...

printf("\nNúmero complejo (%.1f,%.1f), módulo: %.2f",
      pz->pr,pz->pi,pz->modulo);

```

9.3. ESTRUCTURAS ANIDADAS

Una estructura puede contener otras estructuras llamadas *estructuras anidadas*. Las estructuras anidadas ahorran tiempo en la escritura de programas que utilizan estructuras similares. Se han de definir los miembros comunes sólo una vez en su propia estructura y a continuación utilizar esa estructura como un miembro de otra estructura. Consideremos las siguientes dos definiciones de estructuras:

```

struct empleado
{
    char nombre-emp[30];
    char direccion[25];
    char ciudad[20];
    char provincia[20];
    long int cod-postal;
    double salario;
};

struct clientes
{
    char nombre_cliente[30];
    char direccion[25];
    char ciudad[20];
    char provincia[20];
    long int cod-postal;
    double saldo;
};

```

Estas estructuras contienen muchos datos diferentes, aunque hay datos que están solapados. Así, se podría disponer de una estructura, `info_dir`, que contuviera los miembros comunes.

```

struct info-dir
{
    char direccion[25];
    char ciudad[20];
    char provincia[20];
    long int cod-postal;
};

```

Esta estructura se puede utilizar como un miembro de las otras estructuras, es decir, *anidarse*.

```

struct empleado
{
    char nombre-emp[30];
    struct info-dir direccion_emp;
    double salario;
};

struct clientes
{

```

```

char nombre_cliente[30];
struct info-dir direccion_clien;
double saldo;
};

```

Gráficamente se podrían mostrar estructuras anidadas en la Figura 9.2.

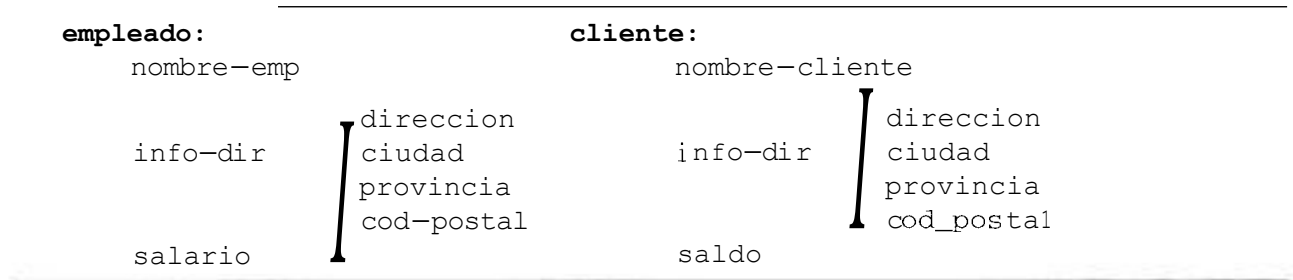


Figura 9.2. Estructuras anidadas.

9.3.1. Ejemplo de estructuras anidadas

Se desea diseñar una estructura que contenga información de operaciones financieras. Esta estructura debe constar de un número de cuenta, una cantidad de dinero, el tipo de operación (depósito=0, retirada de fondos=1, puesta al día=2 o estado de la cuenta=3) y la fecha y hora en que la operación se ha realizado. A fin de realizar el acceso correcto a los campos día, mes y año, así como el tiempo (la hora y minutos) en que se efectuó la operación, se define una estructura fecha y una estructura tiempo. La estructura registro-operacion tiene como miembros una variable (un campo) de tipo fecha, otra variable del tipo tiempo y otras variables para representar los otros campos. La representación del tipo de operación se hace con una variable entera, aunque el tipo apropiado es un tipo enumerado (descrito en siguientes apartados). A continuación se declara estos tipos, se escribe una función que lee una operación financiera y devuelve la operación leída. La fecha y hora es captada del sistema.

```

#include <stdio.h>
#include <dos.h>

struct registro-operation entrada();
struct fecha
{
    unsigned int mes, dia, anyo;
};
struct tiempo
{
    unsigned int horas, minutos;
};
struct registro-operation
{
    long numero-cuenta;
    float cantidad;
    int tipo-operacion;
    struct fecha f;
    struct tiempo t;
};

int main()
{
    struct registro-operation w;

```

```

    w = entrada();

    printf("\n\n Operación realizada\n ");
    printf("\t%ld\n",w.numero_cuenta);
    printf("\t%d-%d-%d\n",w.f.dia,w.f.mes,w.f.anyo);
    printf("\t%d:%d\n",w.t.horas,w.t.minutos);

    return 0;
}

struct registro_operacion entrada()
{
    struct time t;
    struct date d;
    struct registro_operacion una;
    printf("\nNúmero de cuenta: ");
    scanf("%ld",&una.numero_cuenta);
    puts("\n\tTipo de operación");
    puts("Deposito(0)");
    puts("Retirada de fondos(1)");
    puts("Puesta al día(2)");
    puts("Estado de la cuenta(3)");
    scanf("%d",&una.tipo_operacion);

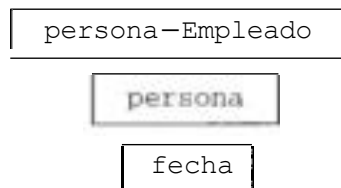
    /* Fecha y tiempo del sistema */
    gettime(&t);
    una.t.horas = t.ti_hour;
    una.t.minutos = t.ti_min;

    getdate(&d);
    una.f.anyo = d.da_year;
    una.f.mes = d.da_mon;
    una.f.dia = d.da-day;
    return una;
}

```

Ejercicio 9.1

Se desea registrar una estructura `PersonaEmpleado` que contenga como miembros los datos de una persona empleado que a su vez tenga los datos de la fecha de nacimiento. En un programa se muestra el uso de la estructura, se define una función para dar entrada a los datos de la estructura y otra función para dar salida a los datos de una estructura persona. A la función de entrada se transmite por dirección (&p) la variable estructura, por lo que el argumento correspondiente tiene que ser un puntero(*p) y el acceso a los campos se hace con el selector ->



```

#include <stdio.h>

struct fecha
{
    unsigned int dia, mes, anyo;
};

```

```

struct persona {
    char nombre[20];
    unsigned int edad;
    int altura;
    int peso;
    struct fecha fec;
};

struct persona-empleado
{
    struct persona unapersona;
    unsigned int salario;
    unsigned int horas_por_semana;
};

/* prototipos de funciones */

void entrada(struct persona-empleado *p);
void muestra(struct persona-empleado up);

void main()
{
    /* define una variable persona-empleado */
    struct persona-empleado p;

    /* llamada a entrada() transmitiendo la direccion */
    entrada(&p);

    /* salida de los datos almacenados */
    muestra(p);
}

void entrada(struct persona-empleado *p)
{
    printf("\nIntroduzca su nombre: ");
    gets(p->unapersona.nombre);
    printf("introduzca su edad: ");
    scanf("%d",&p->unapersona.edad);
    printf("Introduzca su altura: ");
    scanf("%d",&p->unapersona.altura);
    printf("Introduzca su peso: ");
    scanf("%d",&p->unapersona.peso);
    printf("Introduzca su fecha de nacimiento: ");
    scanf("%d %d %d",&p->unapersona.fec.dia,
        &p->unapersona.fec.mes,
        &p->unapersona.fec.anyo);
    printf("Introduzca su salario:");
    scanf("%d",&p->salario);
    printf("introduzca numero de horas:");
    scanf("%d",&p->horas_por_semana);
}

void muestra(struct persona-empleado up)
{
    puts("\n\n\tDatos de un empleado");
    puts("\n\n\t-----");
    printf("Nombre: %s \n",up.unapersona.nombre);
    printf("Edad: %d \n",up.unapersona.edad);
    printf("fecha de nacimiento: %d-%d-%d\n",

```

```

        up.unapersona.fec.dia,
        up.unapersona.fec.mes,
        up.unapersona.fec.anyo);
printf("Altura: %d \n",up.unapersona.altura);
printf("Peso: %d \n",up.unapersona.peso);
printf("Numero de horas: %d \n",up.horas_por_semana);

```

1

El acceso a miembros dato de estructuras anidadas requiere el uso de múltiples operadores punto.

Ejemplo: acceso al día del mes de la fecha de nacimiento de un empleado.

```
up.unapersona.fec.dia
```

Las estructuras se pueden anidar a cualquier grado. También es posible inicializar estructuras anidadas en la definición. El siguiente ejemplo inicializa una variable Luis de tipo `struct persona`.

```
struct persona Luis { "Luis", 25, 1940, 40, {12, 1, 70}};
```

9.4. ARRAYS DE ESTRUCTURAS

Se puede crear un array de estructuras tal como se crea un array de otros tipos. Los arrays de estructuras son idóneos para almacenar un archivo completo de empleados, un archivo de inventario, o cualquier otro conjunto de datos que se adapte a un formato de estructura. Mientras que los arrays proporcionan un medio práctico de almacenar diversos valores del mismo tipo, los arrays de estructuras le permiten almacenar juntos diversos valores de diferentes tipos, agrupados como estructuras.

Muchos programadores de *C* utilizan arrays de estructuras como un método para almacenar datos en un archivo de disco. Se pueden introducir y calcular sus datos de disco en arrays de estructuras y a continuación almacenar esas estructuras en memoria. Los arrays de estructuras proporcionan también un medio de guardar datos que se leen del disco.

La declaración de un array de estructuras `info-libro` se puede hacer de un modo similar a cualquier array, es decir,

```
struct info-libro libros[100];
```

asigna un array de 100 elementos denominado `libros`. Para acceder a los miembros de cada uno de los elementos estructura se utiliza una notación de array. Para inicializar el primer elemento de `libros`, por ejemplo, su código debe hacer referencia a los miembros de `libros[0]` de la forma siguiente:

```
strcpy(libros[0].titulo, "C++ a su alcance");
strcpy(libros[0].autor, "Luis Joyanes");
strcpy(libros[0].editorial, "McGraw-Hill");
libros[0].anyo = 1999;
```

También puede inicializarse un array de estructuras en el punto de la declaración encerrando la lista de inicializadores entre llaves, `{ }`. Por ejemplo,

```
struct info-libro libros[3] = { "C++ a su alcance", "Luis Joyanes",
    "McGraw-Hill", 1999, "Estructura de datos", "Luis Joyanes",
    "McGraw-Hill", 1999, "Problemas en Pascal", "Angel Hermoso",
    "McGraw-Hill", 1997};
```

En el siguiente ejemplo se declara una estructura que representa a un número racional, un array de números racionales es inicializado con valores al azar.

```

struct racional
{
    int N,
    int D;
};
struct racinal rs[4] = { 1,2, 2,3, -4,7, 0,1};

```

9.4.1. Arrays como miembros

Los miembros de las estructuras puede ser asimismo arrays. En este caso, será preciso extremar las precauciones cuando se accede a los elementos individuales del array.

Considérese la siguiente definición de estructura. Esta sentencia declara un array de 100 estructuras, cada estructura contiene información de datos de empleados de una compañía.

```

struct nomina
{
    char nombre[30];
    int dependientes;
    char departamento[10];
    float horas_dias[7];    /* array de tipo float */
    float salario;
} empleado[100];           /* Un array de 100 empleados */

```

Ejemplo 9.1

Una librería desea catalogar su inventario de libros. El siguiente programa crea un array de 100 estructuras, donde cada estructura contiene diversos tipos de variables, incluyendo arrays.

```

#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

struct inventario
{
    char titulo[25];
    char fecha_pub[20];
    char autor[30];
    int num;
    int pedido;
    float precio-venta;
};

int main()
{
    struct inventario libro[100];
    int total = 0;
    char resp, b[21];

    do {
        printf("Total libros %d \n", (total+1));
        printf("¿Cuál es el título?: ");
        gets(libro[total].titulo);

        printf("¿Cuál es la fecha de publicación?: ");
        gets(libro[total].fecha_pub);
    } while (resp != 'n');
}

```

```

printf("¿Quién es el autor?");
gets(libro[total].autor);

printf("¿Cuántos libros existen?: ");
scanf("%d",&libro[total].num);

printf("¿Cuántos ejemplares existen pedidos?: ");
scanf("%d",&libro[total].pedido);

printf("¿Cuál es el precio de venta?: ");
gets(b);
libro[total].precio_venta = atof(b); /* conversión a real */
fflush(stdin);

printf("\n ¿Hay más libros? (S/N)");
scanf("%c",&resp);
fflush(stdin);
resp = toupper(resp); /* convierte a mayúsculas */
if (resp == 'S')
{
    total++;
    continue;
}
} while (resp == 'S');
return 0;

```

9.5. UTILIZACIÓN DE ESTRUCTURAS COMO PARÁMETROS

C permite pasar estructuras a funciones, bien por valor o bien por referencia utilizando el operador &. Si la estructura es grande, el tiempo necesario para copiar un parámetro `struct` a la pila puede ser prohibitivo. En tales casos, se debe considerar el método de pasar la dirección de la estructura.

El listado siguiente muestra un programa que pasa la dirección de una estructura a una función para entrada de datos. La misma variable estructura la pasa por valor a otra función para salida de los campos.

```

#include <stdio.h>

/* Define el tipo estructura info_persona */
struct info-persona {
    char nombre[20];
    char calle[30];
    char ciudad[25];
    char provincia[25];
    char codigopostal[6];
};

/* prototipos de funciones */
void entrad_pna(struct info_persona* pp);
void ver_info(struct info_persona p);

void main (void)
{
    struct info-persona reg_dat;
    /* Pasa por referencia la variable */
    entrad_pna(&reg_dat);
    /* Pasa por valor */
    ver_info(reg_dat);
}

```



```

    printf( "\nPulsa cualquier carácter para continuar\n");
    getchar();
}

void entrada_pna(struct info_persona* pp)
{
    puts("\n Entrada de los datos de una persona\n");
    /* Para acceder a los campos se utiliza el selector -> */
    printf("Nombre: "); gets(pp->nombre);
    printf("Calle: "); gets(pp->calle);
    printf("Ciudad: "); gets(pp->ciudad);
    printf("Provincia: "); gets(pp->provincia);
    printf("Código postal: "); gets(pp->codigopostal);
}

void ver_info(struct info_persona p)
{
    puts("\n\tInformación realativa a la persona");
    puts(p.nombre);
    puts(p.calle);
    puts(p.ciudad);
    puts(p.provincia);
    puts(p.codigopostal);
}

```

Si se desea pasar la estructura por referencia, necesita situar un operador de referencia & antes de reg-dat en la llamada a la función entrada_pna(). El parámetro correspondiente debe de ser tipo puntero struct info_persona* pp. El acceso a miembro de dato de la estructura a partir de un puntero requiere el uso del selector ->.

9.6. UNIONES

Las uniones son similares a las estructuras en cuanto que agrupa a una serie de variables, pero la forma de almacenamiento es diferente y, por consiguiente, efectos diferentes. Una estructura (`struct`) permite almacenar variables relacionadas juntas y almacenadas en posiciones contiguas en memoria. Las uniones, declaradas con la palabra reservada `union`, almacenan también miembros múltiples en un paquete; sin embargo, en lugar de situar sus miembros unos detrás de otros, en una unión, todos los miembros se solapan entre sí en la misma posición. El tamaño ocupado por una unión se determina así: es analizado el tamaño de cada variable de la unión, el mayor tamaño de variable será el tamaño de la unión. La sintaxis de una unión es la siguiente:

```

union nombre {
    tipo1 miembro1;
    tipo2 miembro2;
    ...
};

```

Un ejemplo:

```

union PruebaUnion
{
    float Item1;
    int Item2;
};

```

La cantidad de memoria reservada para una unión es igual a la anchura de la variable más grande. En el tipo `union`, cada uno de los miembros dato comparten memoria con los otros miembros de la unión. La cantidad total de memoria utilizada por la unión `comparte` es de 8 bytes, ya que el elemento `double` es el miembro dato mayor de la unión.

```
union comparte
{
    char letra;
    int elemento;
    float precio;
    double z;
};
```

Una razón para utilizar una unión es ahorrar memoria. En muchos programas se deben tener varias variables, pero no necesitan utilizarse todas al mismo tiempo. Considérese la situación en que se necesitan tener diversas cadenas de caracteres de entrada. Se pueden crear varios arrays de cadenas de caracteres, tales como las siguientes:

```
char linea_ordenes[80];
char mensaje_error[80];
char ayuda[80];
```

Estas tres variables ocupan 240 bytes de memoria. Sin embargo, si su programa no necesita utilizar las tres variables simultáneamente, ¿por qué no permitirle compartir la memoria utilizando una unión? Cuando se combinan en el tipo `union frases`, estas variables ocupan un total de sólo 80 bytes.

```
union frases {
    char linea_ordenes[80];
    char mensaje_error[80];
    char ayuda[80];
} cadenas, *pc;
```

Para referirse a los miembros de una unión, se utiliza el operador punto (`.`), o bien el operador `->` si se hace desde un puntero a unión. Así:

```
cadenas.ayuda;
cadenas.mensaje_error;
pc -> mensaje_error;
```

9.7. ENUMERACIONES

Un **enum** es un tipo definido por el usuario con constantes de nombre de tipo entero. En la declaración de un tipo `enum` se escribe una lista de identificadores que internamente se asocian con las constantes enteras 0, 1, 2, etc.

Formato

1. `enum`

```
{
    enumerador , enumerador , ...enumerador
};
```
2. `enum nombre`

```
{
    enumerador , enumerador , ...enumerador
};
```

En la declaración del tipo `enum` pueden asociarse a los identificadores valores constantes en vez de la asociación que por defecto se hace (0, 1, 2, etc.). Para ello se utiliza este formato:

```
3. enum nombre
{
    enumeradori = expresión_constantei,
    enumerador = expresión-constante,
    ...
    enumeradorn = expresión-constante,
};
```

Ejemplo 9.2

Usos típicos de `enum`

```
enum Interruptor
{
    ENCENDIDO,
    APAGADO
};

enum Boolean
{
    FALSE,
    TRUE
};
```

Ejemplo

```
enum
{
    ROJO, VERDE, AZUL
};
```

define tres constantes `ROJO`, `VERDE` y `AZUL` de valores iguales a 0, 1 y 2, respectivamente. Los miembros datos de un `enum` se llaman enumeradores y la constante entera por defecto del primer enumerador de la lista de los miembros datos es igual a 0. Obsérvese que, al contrario que `struct` y `union`, los miembros de un tipo `enum` se separan por el operador coma. El ejemplo anterior es equivalente a la definición de las tres constantes, `ROJO`, `VERDE` y `AZUL`, tal como:

```
const int ROJO = 0;
const int VERDE = 1;
const int AZUL = 2;
```

En la siguiente declaración de tipo enumerado se le da un nombre al tipo

```
enum dias-semana
{
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
};
```

Una variable de tipo `enum dias-semana` puede tomar los valores especificados en la declaración del tipo. El siguiente bucle está controlado por una variable del tipo enumerado.

```
enum dias-semana dia;

for (dia = LUNES; dia <= DOMINGO; dia++)
{
    ...
}
```

```
    printf("%d ",dia);
}
```

La ejecución del bucle escribiría en pantalla: 0 1 2 3 4 5 6.

A los enumeradores se pueden asignar valores constantes o expresiones constantes durante la declaración:

```
enum Hexaedro
{
    VERTICES = 8,
    LADOS    = 12,
    CARAS    = 6
}
```

Ejercicio 9.2

El siguiente programa muestra el uso *de* la enumeración *boolean*. El programa lee un texto y cuenta las vocales leídas. La función `vocal()` devuelve TRUE si el carácter de entrada es vocal.

```
#include <stdio.h>
#include <ctype.h>

enum boolean
{
    FALSE, TRUE
};

enum boolean vocal(char c);

void main()
{
    char car;
    int numvocal = 0;

    puts("\nIntroduce un texto. Para terminar: INTRO ");
    while ( (car = getchar()) != '\n')
    {
        if (vocal(tolower(car)))
            numvocal++;
    }
    printf("\n Total de vocales leídas: %d\n",numvocal);
}

enum boolean vocal(char c)

{
    switch (c)
    {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            return TRUE;
        default:
            return FALSE;
    }
}
```

9.7.1. sizeof de tipos de datos estructurados

El tamaño en bytes de una estructura, de una unión o de un tipo enumerado se puede determinar con el operador `sizeof`.

El siguiente programa extrae el tamaño de una estructura (`struct`), de una unión (`union`) con miembros dato idénticos, y de un tipo enumerado.

```
/* declara una union */
union tipo-union
{
    char c;
    int i;
    float f;
    double d;
};

/* declara una estructura */
struct tipo-estructura
{
    char c;
    int i;
    float f;
    double d;
};

/* declara un tipo enumerado */
enum monedas
{
    PESETA,
    DURO,
    CINCODUROS,
    CIEN
};
...

printf("\nsizeof (tipo-estructura): %d\n",
        sizeof(struct tipo-estructura));
printf("\nsizeof (tipo-union): %d\n",
        sizeof(union tipo-union));
printf("\nsizeof (monedas): %d\n",
        sizeof(enum monedas));
```

La salida que se genera con estos datos:

```
sizeof(tipo_estructura):15
sizeof(tipo_union): 8
sizeof(monedas): 2
```

9.7.2. typedef

Un **typedef** permite a un programador crear un sinónimo de un tipo de dato definido por el usuario o de un tipo ya existente.

Ejemplo

Uso de typedef para declarar un nuevo nombre, Longitud, de tipo de dato double.

```

...
typedef double Longitud;
...
Longitud Distancia (const struct Pto* p, const struct Pto* p2)
{
    ...
    Longitud longitud = sqrt(r-cua);
    return longitud;
}

```

Otros ejemplos:

```

typedef char* String;
typedef const char* string;

```

Puede declararse un tipo estructura o un tipo unión y a continuación asociar el tipo estructura a un nombre con *typedef*.

Ejemplo

Declaración del tipo de dato complejo y asociación a complex.

```

struct complejo
{
    float x,y;
};

typedef struct complejo complex;

/* definición de un array de complejos */
complex v[12];

```

La ventaja de typedef es que permite dar nombres de tipos de datos más acordes con lo que representan en una determinada aplicación.

9.8. CAMPOS DE BIT

El lenguaje C permite realizar operaciones con los bits de una palabra. Ya se han estudiado los operadores de manejo de bits: >>, <<, ... Con los campos de bits, C permite acceder a un número de bits de una palabra entera. Un campo de bits es un conjunto de bits adyacentes dentro de una palabra entera. La sintaxis para declarar campos de bits se basa en la declaración de estructuras. El formato general:

```

struct identificador-campo {
    tipo nombre1: longitud1;
    tipo nombre2: longitud2;
    tipo nombre3: longitud3;

    tipo nombren: longitudn;
};

```

tipo ha de ser entero, `int`; generalmente `unsigned int`
 longitud es el número de bits consecutivos que se toman

Ejemplo 9.3

En este ejemplo se declara un campo de bits para representar en formato comprimido el día, mes año (los dos últimos dígitos) y si el año es bisiesto.

```
struct fecha {
    unsigned dia: 5;
    unsigned mes: 4;
    unsigned año: 7;
    unsigned bisiesto: 1;
};
```

Ejemplo 9.4

El siguiente ejemplo muestra cómo puede utilizarse campos de bits para representar si están o no conectados diversos componentes eléctricos. Cada componente se representa con un flag, con un bit; cuando esté puesto a cero es que no está conectado, cuando esté puesto a uno está conectado.

```
struct componentes {
    unsigned diodo: 1;
    unsigned resistencia: 1;
    unsigned amperimetro: 1;
    unsigned transistor: 1;
    unsigned condensador: 1;
    unsigned inductancia: 1;
};
```

Los campos individuales se referencian como cualquier otro miembro de una estructura: selector punto (`.`). Por ejemplo,

```
struct componentes ct;
ct.diodo = 1;
if (ct.amperimetro)
{
}
```

Al declarar campos de bits, la suma de los bits declarados puede exceder el tamaño de un entero; en ese caso se emplea la siguiente posición de almacenamiento entero. No está permitido que un campo de bits solape los límites entre dos `int`.

Al declarar una estructura puede haber miembros que sean variables y otros campos de bits. La siguiente estructura tiene esta característica:

```
struct reg-estudiante{
    char nombre [33];
    char apell1 [33];
    char apell2 [33];
    unsigned masculino: 1;
    unsigned femenino: 1;
    unsigned curso: 3;
};
```

Los campos de bits se utilizan para rutinas de encriptación de datos y fundamentalmente para ciertos interfaces de dispositivos externos. Presentan ciertas restricciones. Así, no se puede tomar la dirección de una variable campo de bits; no puede haber arrays de campos de bits; no se puede solapar fronteras de `int`. Depende del procesador que los campos de bits se alineen de izquierda a derecha o de derecha a izquierda (conviene hacer una comprobación para cada procesador, utilizando para ello un `union` con variable entera y campos de bits).

Ejemplo 9.5

Se tiene la función `peticion_acceso()` capaz de direccionar una posición de memoria de 8 bits si recibe como argumento una variable llamada `ochobits`. Con esta variable controla a través de cada bit las peticiones de acceso a cada uno de los ocho periféricos distintos con que trabaja; eventos externos son los que se encargan de cargar la variable `ochobits`.

Se quiere escribir una función que determine cuantos accesos se producen por cada periférico en un bucle de 1000 llamadas a la función `peticion_acceso()`. Se supone que cada llamada sólo activa un periférico.

Análisis

El tipo de la variable `ochobits` va a ser una estructura de campos de bits, cada campo con longitud 1; por lo que puede tener dos estados, 0 o 1, para indicar no acceso o sí acceso. Un array de 8 elementos, tantos como periféricos se utiliza para contar los accesos a cada periférico.

```
/* Tipo estructura de campos de bits */

struct perifericos{
    unsigned perf1: 1;
    unsigned perf2: 1;
    unsigned perf3: 1;
    unsigned perf4: 1;
    unsigned perf5: 1;
    unsigned perf6: 1;
    unsigned perf7: 1;
    unsigned perf8: 1;
};

/* Prototipo de función peticion_acceso() */
void peticion-acceso (const struct perifericos ochobits);

/* Función que contabiliza los accesos a cada periférico. */

void accesos_perf(int acceper[])
{
    int i;
    const struct perifericos ochobits;
    const neventos=1000;
    for (i=0; i<8; )
        acceper[i++] = 0;

    /* Bucle principal de 1000 llamadas */
    for (i=0; i<neventos; i++)
    {
```



```

peticion_acceso(ochobits);
if (ochobits.perf1)
    ++acceper[0];
elseif (ocho.bits.perf2);
    ++acceper[1];
elseif (ocho.bits.perf3);
    ++acceper[2];
elseif (ocho.bits.perf4);
    ++acceper[3];
elseif (ocho.bits.perf5);
    ++acceper[4];
elseif (ocho.bits.perf6);
    ++acceper[5];
elseif (ocho.bits.perf7);
    ++acceper[6];
elseif (ocho.bits.perf8);
    ++acceper[7];

```

Ejemplo 9.6

Haciendo uso de una estructura de campo de bits y de una union, en este ejercicio se escribe un programa para visualizar la decodificación en bits de cualquier carácter leído por teclado.

Análisis

Se declara un tipo estructura campo de bits, con tantos campos como bits tiene un byte, que a su vez es el almacenamiento de un carácter. La decodificación es inmediata declarando una union entre una variable carácter y una variable campo de bits del tipo indicado.

```

#include <stdio.h>
#include <conio.h>

#define mayus (ch) ((ch>='a' && ch<='z') ? (ch+'A'-'a') : ch)

struct byte {
    unsigned int a: 1;
    unsigned int b: 1;
    unsigned int c: 1;
    unsigned int d: 1;
    unsigned int e: 1;
    unsigned int f: 1;
    unsigned int g: 1;
    unsigned int h: 1;
};

union charbits{
    char ch;
    struct byte bits;
}caracter;

void decodifica (struct byte b);

```

```

void main()
{
    puts ('Teclea caracteres. Para salir carácter X');
    do {
        caracter.ch = getche();
        printf(" : ");
        decodifica(caracter.bits);
    }while mayusc(caracter.ch) != 'X';
}

void decodifica(struct byte b)

/* Los campos de bits se alinean de derecha a izquierda, por esa razón se
escriben los campos en orden inverso */
printf("%2u%2u%2u%2u%2u%2u%2u%2u \n",
        b.h, b.g, b.f, b.e, b.d, b.c, b.b, b.a);
}

```

9.9. RESUMEN

Una estructura permite que los miembros dato de los mismos o diferentes tipos se encapsulen en una única implementación, al contrario que los arrays que son agregados de un tipo de dato simple. Los miembros dato de una estructura son accesibles con el operador punto(.), o con el operador flecha(->).

- Una estructura es un tipo de dato que contiene elementos de tipos diferentes.

```

struct empleado {
    char nombre[30];
    long salario;
    char num_telefono[10];
};

```

- Para crear una variable estructura se escribe

```
struct empleado pepe;
```

- Para determinar el tamaño en bytes de cualquier tipo de dato C utilizar el operador sizeof.
- El tipo de dato union es similar a struct de modo que es una colección de miembros dato de tipos diferentes o similares, pero al contrario que una definición struct, que asigna memoria suficiente para contener todos los miembros dato, una union puede contener sólo un miembro dato en cualquier momento dado y el tamaño de una unión es, por consiguiente, el tamaño de su miembro mayor.
- Utilizando un tipo union se puede hacer que diferentes variables coexistan en el mismo espacio de memoria. La unión permite reducir espacio de memoria en sus programas.
- Un typedef no es nuevo tipo de dato sino un sinónimo de un tipo existente.

```

typedef struct empleado
regempleado;

```

9.10. EJERCICIOS

- 9.1. Encuentra los errores en la siguiente declaración de estructura y posterior definición de variable.

```
struct hormiga
{
    int patas;
    char especie[41];
    float tiempo;
} hormiga colonia[100];
```

- 9.2. Declara un tipo de datos para representar estaciones del año.
- 9.3. Escribe un función que devuelva la estación año que se ha leído del teclado. La función debe ser del tipo declarado en el Ejercicio 9.2.
- 9.4. Declara un tipo de dato enumerado para representar los meses del año, el mes enero debe estar asociado al dato entero 1, y así sucesivamente los demás meses.
- 9.5. Encuentra los errores del siguiente código

```
#include <stdio.h>
void escribe(struct fecha f)
int main()
{
    struct fecha
    {
        int dia;
        int mes;
        int anyo;
        char mes[];
```

```
} ff;
ff = {1,1,2000,"ENERO"};
escribe(ff);
return 1;
}
```

9.6. Con ayuda de la declaración anterior, escribe una función que devuelva la estación del año que se ha leído del teclado.

9.11. PROBLEMAS

9.1. Escribe un programa para calcular el número de días que hay entre dos fechas; declarar fecha como una estructura.

9.2. Escribe un programa de facturación de clientes. Los clientes tienen un nombre, el número de unidades solicitadas, el precio de cada unidad y el estado en que se encuentra: moroso, atrasado, pagado. El programa debe generar a los diversos clientes.

9.3. Modifique el programa de facturación de clientes de tal modo que se puedan obtener los siguientes listados.

- Clientes en estado moroso.
- Clientes en estado pagado con factura mayor de una determinada cantidad.

9.4. Escribe un programa que permita hacer las operaciones de suma, resta, multiplicación y división de números complejos. El tipo complejo ha de definirse como una estructura,

9.5. Un número racional se caracteriza por el numerador y denominador. Escribe un programa para operar con números racionales. Las operaciones a definir son la suma, resta, multiplicación y división; además de una función para simplificar cada número racional.

9.6. Se quiere informatizar los resultados obtenidos por los equipos de baloncesto y de fútbol de la localidad alcarreña Lupiana. La información de cada equipo:

- Nombre del equipo.
- Número de victorias.
- Número de derrotas.

Para los equipos de baloncesto añadir la información:

- Número de pérdidas de balón.
- Número de rebotes cogidos.
- Nombre del mejor anotador de triples.
- Número de triples del mejor triplista.

Para los equipos de fútbol añadir la información:

- Número de empates.
- Número de goles a favor.
- Número de goles en contra.
- Nombre del goleador del equipo.
- Número de goles del goleador.

Escribir un programa para introducir la información para todos los equipos integrantes en ambas ligas.

9.7. Modificar el programa 9.6 para obtener los siguientes informes o datos.

- Listado de los mejores triplistas de cada equipo.
- Máximo goleador de la liga de fútbol.
- Suponiendo que el partido ganado son tres puntos y el empate 1 punto: equipo ganador de la liga de fútbol.
- Equipo ganador de la liga de baloncesto.

9.8. Un punto en el plano se puede representar mediante una estructura con dos campos. Escribir un programa que realice las siguientes operaciones con puntos en el plano.

- Dados dos puntos calcular la distancia entre ellos.
- Dados dos puntos determinar la ecuación de la recta que pasa por ellos.
- Dados tres puntos, que representan los vértices de un triángulo calcular su área.